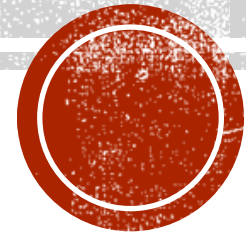




**Faculty of Economics and
Business Administration**

WEEK 12

How to Access REST API



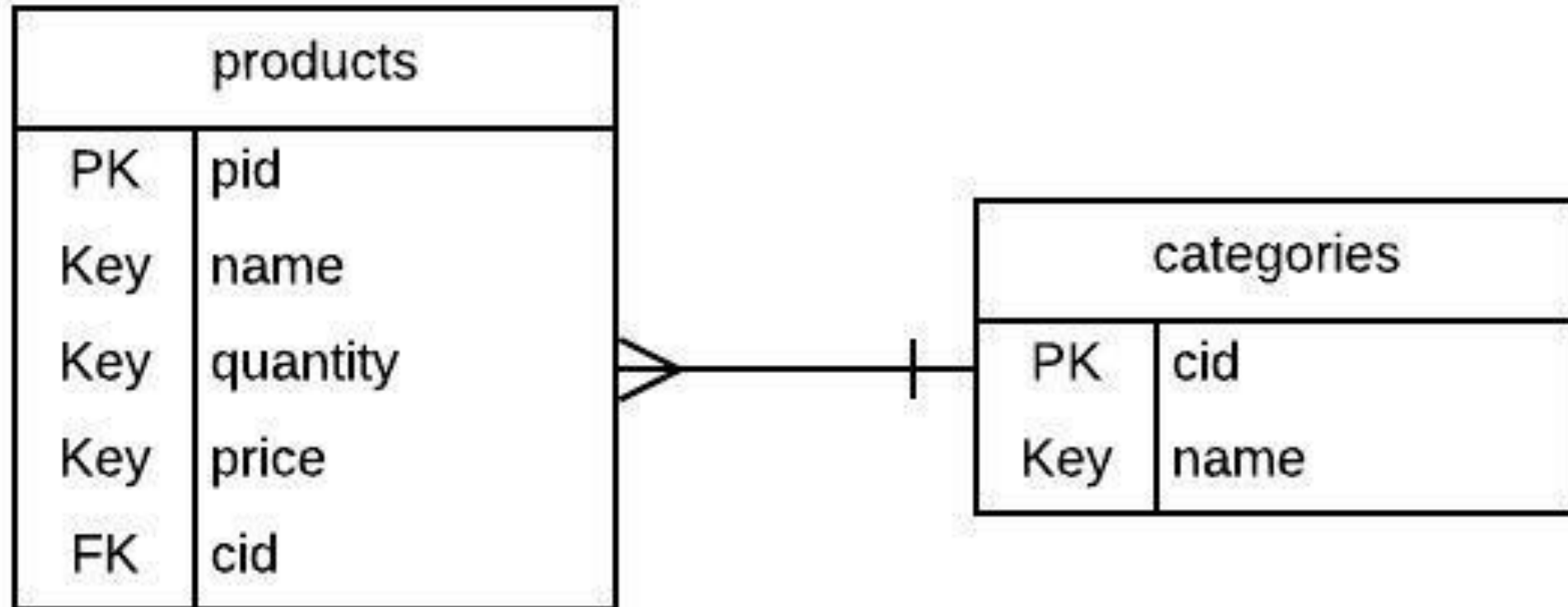
Dr. Abdul Rahman EL SAYED

PRESENTATION TOPICS:

- In this presentation, we will learn the following topics:
 - Use asynchronous functions
 - Retrieve and manipulate JSON arrays
 - Populate a ListView with data from a REST service



WE WILL USE THE BELOW DATABASE



WE WILL USE THE BELOW REST SERVICE

/public_html/getProducts.php

```
1 <?php
2 header("Cache-Control: no-store, no-cache, must-revalidate, max-age=0");
3 header("Cache-Control: post-check=0, pre-check=0", false);
4 header("Pragma: no-cache");
5
6 $con=mysqli_connect("localhost","id21571399_user1", "password","id21571399_university");
7 // Check connection
8 if (mysqli_connect_errno())
9 {
10     echo "Failed to connect to MySQL: " . mysqli_connect_error();
11 }
12
13 $sql = "select products.pid, products.name, products.quantity, products.price, categories.name category
14 from products
15 INNER JOIN categories on products.cid = categories.cid";
16 if ($result = mysqli_query($con,$sql))
17 {
18     $emparray = array();
19     while($row =mysqli_fetch_assoc($result))
20     |     $emparray[] = $row;
21
22     echo(json_encode($emparray));
23     // Free result set
24     mysqli_free_result($result);
25     mysqli_close($con);
```

JSON FORMAT

- ❑ Since we can work with multiple DBMS and each DBMS has a different format to represent data, we need a standard format to send/receive data on the net.
- ❑ This format is called JSON, most modern languages have support for JSON data.
- ❑ Our REST service will return the products as a JSON array.

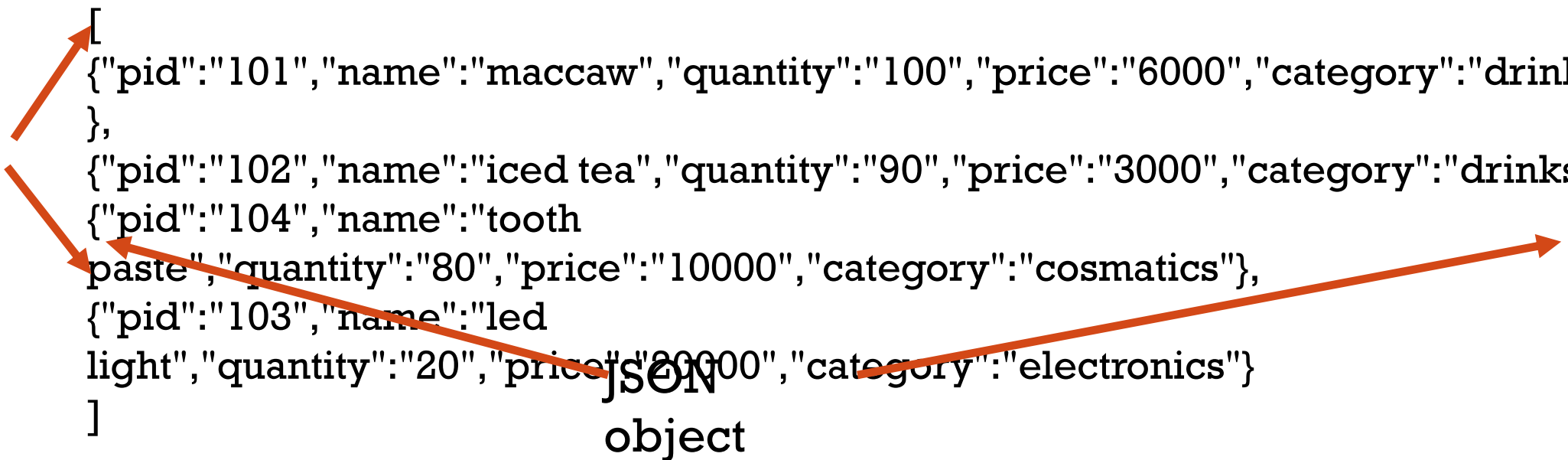


JSON DATA EXAMPLE

JSON array

```
[  
  {"pid":"101","name":"maccaw","quantity":"100","price":"6000","category":"drinks"  
  },  
  {"pid":"102","name":"iced tea","quantity":"90","price":"3000","category":"drinks"},  
  {"pid":"104","name":"tooth  
paste","quantity":"80","price":"10000","category":"cosmetics"},  
  {"pid":"103","name":"led  
light","quantity":"20","price":"20000","category":"electronics"}  
]
```

JSON object



THE APPLICATION

- We will create an application that retrieves the products from the REST service and displays it in a ListView widget.



ASYNCHRONOUS FUNCTIONS

- ❑ Since getting information from the net, is a time consuming process, we will use asynchronous functions.
- ❑ Asynchronous functions will not block the execution of the application and keeps it responsive. You declare asynchronous functions using the `async` keyword.
- ❑ You wait for a long executing function inside an `async` function, using the `wait` keyword.



ASYNCHRONOUS FUNCTIONS EXAMPLE

```
import 'dart:convert' as convert;

import 'package:http/http.dart' as http;

void main(List<String> arguments) async {
  // This example uses the Google Books API to search
  // for books about HTTP. For details, see
  // https://developers.google.com/books/docs/overview
  final url = Uri.https(
    'www.googleapis.com',
    '/books/v1/volumes',
    {'q': '{http}'},
  );

  // Await the HTTP GET response, then decode the
  // JSON data it contains.
  final response = await http.get(url);

  if (response.statusCode == 200) {
    final jsonResponse = convert.jsonDecode(response.body);
    final itemCount = jsonResponse['totalItems'];
    print('Number of books about HTTP: $itemCount.');
  } else {
    print('Request failed with status: ${response.statusCode}.');
  }
}
```

THE HTTP PACKAGE

- ❑ We will use the HTTP package to receive data from REST pages.
- ❑ You need to add the package to your project.
- ❑ The easiest way is to open the terminal after you created the project and execute the below commands:

flutter pub add http

Flutter pub get



THE HTTP PACKAGE

After you execute the commands, you will see the below lines in your pubspec.yaml file:

```
# The following adds the Cupertino Icons font to your application.  
# Use with the CupertinoIcons class for iOS style icons.  
cupertino_icons: ^1.0.2  
http: ^1.1.0  
  
dev_dependencies:  
  flutter_test:  
    sdk: flutter
```

Automatically inserts the
latest HTTP package
version



CREATING A CLASS TO REPRESENT A PRODUCT IN PRODUCT.DART

```
import 'dart:convert' as convert;
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
// main URL for REST pages
const String _baseUrl = 'csci410fall2023.000webhostapp.com';

// class to represent a row from the products table
// note: cid is replaced by category name
class Product {
  int _pid;
  String _name;
  int _quantity;
  double _price;
  String _category;

  Product(this._pid, this._name, this._quantity, this._price, this._category);

  @override
  String toString() {
    return 'PID: $_pid Name: $_name Quantity: $_quantity Price: \$_price Category: $_category';
  }
}
```

CREATING AN ASYNCHRONOUS FUNCTION TO GET DATA

```
// list to hold products retrieved from getProducts
List<Product> _products = [];
// asynchronously update _products list
void updateProducts(Function(bool success) update) async {
  try {
    final url = Uri.https(_baseUrl, 'getProducts.php');
    final response = await http.get(url)
      .timeout(const Duration(seconds: 5)); // max timeout 5 seconds
    _products.clear(); // clear old products
    if (response.statusCode == 200) { // if successful call
      final jsonResponse = convert.jsonDecode(response.body); // create dart json object from json array
      for (var row in jsonResponse) { // iterate over all rows in the json array
        Product p = Product( // create a product object from JSON row object
          int.parse(row['pid']),
          row['name'],
          int.parse(row['quantity']),
          double.parse(row['price']),
          row['category']);
        _products.add(p); // add the product object to the _products list
      }
      update(true); // callback update method to inform that we completed retrieving data
    }
  }
  catch(e) {
    update(false); // inform through callback that we failed to get data
  }
}
```

CREATING A WIDGET TO DISPLAY PRODUCTS

```
// shows products stored in the _products list as a ListView
class ShowProducts extends StatelessWidget {
  const ShowProducts({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    double width = MediaQuery.of(context).size.width;
    return ListView.builder(
      itemCount: _products.length,
      itemBuilder: (context, index) => Column(children: [
        const SizedBox(height: 10),
        Container(
          color: index % 2 == 0 ? Colors.amber : Colors.cyan,
          padding: const EdgeInsets.all(5),
          width: width * 0.9, child: Row(children: [
            SizedBox(width: width * 0.15),
            Flexible(child: Text(_products[index].toString(), style: TextStyle(fontSize: width * 0.045)))
          ])) // Row, Container
        ]) // Column
    ); // ListView.builder
  }
}
```

THE INITSTATE METHOD

- ❑ The initState method is called only once when the widget is first added to the tree.
- ❑ We will use it to load the data from the REST service, when the application starts. It is found in the home.dart file and its code is

```
@override
void initState() {
  // update data when the widget is added to the tree the first time.
  updateProducts(update);
  super.initState();
}
```

THE UPDATE CALLBACK METHOD

- ❑ The update method is found in the home.dart file.
- ❑ It is called by the asynchronous updateProducts method, when it completes retrieving data. Its code is below:

```
class _HomeState extends State<Home> {  
  bool _load = false; // used to show products list or progress bar  
  
  void update(bool success) {  
    setState(() {  
      _load = true; // show product list  
      if (!success) { // API request failed  
        ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('failed to load data')));  
      }  
    });  
  }  
}
```


THE HOME.DART PAGE GUI

- ❑ The body property will display a progress bar when loading data.
- ❑ It will then display a ListView of products when the data is retrieved from the getProducts.php page.

```
title: const Text('Available Products'),  
centerTitle: true,  
, // AppBar  
// load products or progress bar  
body: _load ? const ShowProducts() : const Center(  
  child: SizedBox(width: 100, height: 100, child: CircularProgressIndicator())  
) // Center  
); // Scaffold  
}
```